

CSD358: IR Hackathon — Midsem assignment

Overview

Teams of 3-4 members will get 36 hours to build a working IR system on one of six given tracks. Grading rewards how well you use IR principles from your lectures and how original the idea is.

You do not need a finished product. A working partial implementation is enough, as long as the part that works uses IR principles from the lectures covered so far. Teams can continue developing this hackathon project as their CSD358 course project.

Item	Detail
Deadline	1.5 days (36 hours) after the tracks are announced
Tracks	6; each team can pick any one
Scope	A working prototype or partial implementation is enough
Continuation	Optional: You can carry the project forward as your course project, or you can pick something else and start from scratch
Output	Code, a demo video with explanation and a report that includes the work division and an AI-use declaration
Presentation (no viva)	Online (through a pre recorded video)
Weightage	25% of the course grade

IR principles to build on

Every project must use relevant IR principles from the lectures below. You are free to go beyond IR: ML, NLP, LLMs, systems, etc. All ideas are welcome as long as they support the project. IR must still be a real part of how the system works, not just a label on it. Here are a few concepts which you can include in your project.

Topic	Concepts you can build on
Boolean retrieval	Inverted index (dictionary + postings), term-document incidence matrix, AND/OR/NOT query processing, postings intersection, query optimisation (process terms in order of increasing

Topic	Concepts you can build on
	document frequency)
Term vocabulary and postings	Deciding what a document is, tokenization, normalization, case folding, stop words, stemming (Porter), lemmatization, Soundex, non-English text, skip pointers, biword and positional indexes, phrase queries, precision and recall
tf-idf and the vector space model	Jaccard coefficient, bag of words, log-frequency tf weighting, document frequency and idf, tf-idf, documents and queries as vectors, length normalization, cosine similarity, SMART weighting schemes (e.g. Inc.Itc)
Scoring and result assembly	Efficient cosine scoring, heap-based top-K selection, index elimination (high-idf terms, many-term matching), champion lists, static quality g(d) and net score, impact-ordered postings, high/low lists, tiered indexes, cluster pruning (leaders and followers), zone and parametric indexes, query term proximity, query parsers, putting together a complete search system
Web crawling	Basic crawl loop, URL frontier, politeness, robots.txt, URL normalisation, the "content seen?" check for duplicates, spider traps, freshness and recrawling, Mercator front and back queues, distributed crawling

Notes:

- There is no minimum number of concepts. Use the ones that fit your idea and use them properly.
- Libraries and tools are allowed; explain what they do in your pipeline in IR terms in your report.
- IR Methods outside the syllabus are welcome, for example, **BM25, dense retrieval, learning-to-rank, or PageRank. (For extra points)**

The six tracks

Each track is modelled on a research theme that features prominently at SIGIR, WSDM or WWW (The Web Conference). The example ideas are starting points, not specifications, and building one exactly as written will score low on novelty. You can use any public dataset or one you crawl yourself, as long as you respect robots.txt.

T1	Retrieval-Augmented Generation and trustworthy answers
T2	Conversational and agentic search
T3	Recommender systems and personalization
T4	Web crawling, freshness and web integrity
T5	Multilingual and Indic-language search
T6	Vertical search for law, finance or science

T1: Retrieval-Augmented Generation (RAG) and trustworthy answers

Theme: An LLM is only as good as what gets retrieved for it. The research focus has moved to retrieval quality, getting answers to cite their sources, and catching hallucinations.

Challenge: Build a RAG system where the retriever is a real IR component that you can inspect, and where every generated claim can be traced to a ranked source.

IR hooks: chunking as the "what is a document" decision, an inverted index plus tf-idf/cosine retriever, top-K with a heap or champion lists, zone weighting (e.g. title vs body), and precision@k of the retriever.

Sample Ideas: a hybrid of sparse (tf-idf) and dense retrieval that shows when each one wins; a citation checker that re-scores each generated sentence against its cited chunk with cosine similarity and flags unsupported claims.

Sample Data: MS MARCO passages, Natural Questions, BEIR subsets (SciFact, FiQA), or a corpus you build yourselves.

T2: Conversational and agentic search

Theme: Search is becoming a dialogue and, increasingly, an agent that issues many queries on the user's behalf ("deep research" systems).

Challenge: Build a multi-turn search system that keeps track of context and rewrites or decomposes queries before it retrieves.

IR hooks: query rewriting and expansion using vocabulary normalisation, stemming and synonyms; Boolean query construction by an agent; vector-space ranking over the rewritten query; query parsers that turn one request into several sub-queries and merge the scores.

Sample Ideas: a resolver for follow-up questions ("what about its price?") that shows each rewritten query and its ranked results; a research agent that decomposes a question into Boolean and free-text sub-queries and fuses the ranked lists; a clarifying-question generator driven by high-idf terms that the query leaves ambiguous.

Sample Data: TREC CAsT or TREC iKAT topics, QReCC, etc.

T3: Recommender systems and personalization

Theme: Recommendation is at the centre of WSDM. Current topics include cold start, explaining recommendations, diversity, and LLM-based recommenders.

Challenge: Build a recommender that treats items and user profiles as documents and queries in a vector space, and can explain why it recommended each item.

IR hooks: item and user tf-idf vectors with cosine similarity; Jaccard for set-based similarity; static quality scores and net score combining relevance with popularity or recency; cluster pruning for fast candidate generation; champion lists per user interest.

Sample Ideas: cold-start news recommendations from article text alone; "because you read X" explanations built from the terms that contribute most to the cosine; re-ranking for diversity, trading off net score against coverage; a course or elective recommender for SNU students built from syllabus text.

Sample Data: MIND (Microsoft News), MovieLens with plot summaries, Amazon Reviews, Goodreads.

T4: Web crawling, freshness and web integrity

Theme: WWW keeps coming back to the health of the web: duplicates, spam, misinformation, crawling that respects sites, and keeping an index fresh.

Challenge: Build a focused crawler plus a small index that addresses one integrity problem end to end.

IR hooks: URL frontier, politeness and robots.txt; URL normalisation and the "content seen?" check; detecting near-duplicates with Jaccard over shingles; a Mercator-style priority frontier; indexing and ranking what you crawled.

Sample Ideas: a crawler for news or government notices that detects near-duplicate and rewritten articles; a scheduler that decides when to recrawl each page from how often it has changed before; a spider-trap and spam detector with a live frontier dashboard; a claim-tracking crawler that finds every page repeating a claim and ranks pages by source quality $g(d)$.

Sample Data: pages you crawl yourselves (seeds you choose, obeying robots.txt), Common Crawl samples, FEVER for claims.

T5: Multilingual and Indic-language search

Theme: Searching in languages with few resources, and in code-mixed text, is a growing research area. India is an obvious testbed: Hinglish, transliteration, and 22 scheduled languages.

Challenge: Build a search system that works for at least one Indian language, or for code-mixed or transliterated queries.

IR hooks: tokenisation and normalisation for non-English scripts; stemmers or lemmatisers for Indic languages, compared with no stemming; Soundex-style phonetic matching for transliterated spellings; how stop words and idf behave on a Hindi corpus; cross-lingual ranking with the vector space model.

Sample Ideas: a Hinglish query engine where "kal ka mausam", "कल का मौसम" and "weather tomorrow" return the same results; a phonetic matcher for spelling variants of names across scripts; a regional-news search engine that reports precision and recall with and without stemming.

Sample Data: MIRACL (includes Hindi, Bengali and Telugu), FIRE shared-task collections, AI4Bharat IndicCorp, Hindi Wikipedia.

T6: Vertical search for law, finance or science

Theme: Domain search engines are a large share of industrial IR work. They rely on structure: zones, metadata, citations and authority.

Challenge: Build a search engine for one professional domain that uses the domain's document structure, not just the flat text.

IR hooks: zone and parametric indexes (e.g. court, year, section, or company and filing type); Westlaw-style Boolean and proximity queries; positional indexes for exact phrases such as statute names; static authority scores and net score; tiered indexes.

Sample Ideas: a legal precedent finder that ranks judgments by tf-idf plus citation authority; an SEC or SEBI filing search with zone queries ("risk factors" section only); an arXiv search that pulls up the papers a given abstract should cite; a company-annual-report QA that combines Boolean filters with cosine ranking.

Sample Data: Indian Kanoon (crawled politely) or the ILDC dataset, SEC EDGAR filings, the arXiv metadata dump, TREC-COVID / CORD-19.

Deliverables

S.No	Deliverable	Format and limits
1	Code	GitHub repo link with a README covering setup, how to run it, where the data comes from, and what works and what is still planned
2	Demo video with explanation	5–8 mins, unlisted YouTube or Drive link. Show the working part running live, No slides . Team members explain the

S.No	Deliverable	Format and limits
		work.
3	Report	PDF, at most 8 pages excluding references and appendix, using the structure below

What the video must show

1. The problem, and why it belongs to the chosen track (≤ 1 min).
2. The system running end to end on real queries or inputs, including at least one limitation.
3. The pipeline, then evaluation. Show the actual code or intermediate output (postings, weights, scores), not only the final screen.
4. The evaluation results: P/R/P@k against a baseline, or another justified evaluation technique.
5. Each member explaining the component they own.

Report structure

1. **Problem and track relevance:** Any research papers referred, the user needs, and why it matters to the track's research theme.
2. **How you used IR:** which IR principles you used, where they are in the code and why you chose them. Include a pipeline diagram.
3. **Beyond IR (if any):** other techniques you used and how they support the IR side.
4. **Novelty and creativity:** what is new compared with the obvious baseline or existing tools.
5. **Evaluation:** a small set of queries you judged yourselves, precision/recall/P@k, and a comparison with a simple baseline (or any other justified evaluation technique). (graphs, tabular representations would be preferred)
6. **Limitations and next steps. If you plan to continue this as your course project, add a short roadmap.**
7. **Work Division should also be there in the report**

Work division and AI use

Work division: A short note on who worked on what. It is for information only. There are no percentages and no marks for it.

AI-use declaration: There is no fixed format. Declare wherever you used AI in the report.

Rules

- **Frontend is not graded.** A CLI, a Jupyter notebook, a Streamlit page or a full web app all earn the same marks. Do not spend the 2 days on UI.

- **Partial implementations are fine.** You do not need a complete final project. A prototype or a working part of the system is sufficient, as long as it runs and uses the IR principles covered so far. A demo that is faked, hard-coded or shown only as screenshots earns 0 for "working system".
- **AI use is allowed and not penalised.** This includes AI coding agents such as Claude Code, Cursor, Copilot or Codex, which may write code for you. Declare where you used them. **Thinking outside IR is encouraged**, as long as relevant IR principles are part of the system.
- **You can continue the project.** Teams may carry this work forward as their CSD358 course project.
- **Build it within the 1.5-day window.** Do not submit an earlier project.
- **No copying of existing work.** Any submission found to be a direct copy of work already published online (GitHub repos, papers, blogs, tutorials) or of previous projects within SNU's domain will be dealt with strictly under the university's academic integrity policy. Using public libraries, datasets and AI help is fine when declared. Submitting someone else's project as your own is not.
- **Data ethics.** Obey robots.txt, add delays between requests to the same host, and do not collect personal data. Credit every dataset.

Evaluation rubric (100 marks)

Criterion	Marks	Full marks
Use of IR principles	30	Relevant IR principles from the lectures, used correctly, with the choices explained; techniques from outside IR integrate well with them
Novelty and creativity	10	A clear idea that is new compared with the obvious baseline and with existing tools
Working system	20	The implemented part (it can be partial) runs live on real inputs, and the README reproduces it
Evaluation metrics	15	A few judged queries with P/R/P@k and a comparison with a baseline, or a clear sanity check for a partial system
Relevance to the track	5	Directly addresses the track's research theme
Report quality	10	Clearly explains how IR was used, why it matters, what is

Criterion	Marks	Full marks
		novel and what comes next
Video explanation	10	Clear demo and explanation of the system
Total	100	

Submission checklist and FAQ

Submit one form per team at the provided form link within the deadline.

- ☐ Repo link, with a README that explains how to run the system
- ☐ Demo video link (5–8 mins, unlisted on YouTube or Drive)
- ☐ Report PDF (≤ 8 pages), including the work division and AI-use declaration

FAQ

- **Does it have to be a finished project?** No. A working partial implementation that uses IR principles is enough.
- **Can we continue this as our course project?** Yes. The hackathon version can be the first milestone.
- **Can we use AI coding agents?** Yes, including agents that write most of the code. Mention them in your declaration. There is no deduction.
- **Can we use pretrained models or LLM APIs?** Yes. Declare them and ensure IR principles remain part of the system.
- **Can the project be mostly ML or NLP?** Yes, if IR principles play a relevant role.
- **Do we need to deploy it online?** No. Running it locally in the video is enough.
- **How big should the corpus be?** Big enough that ranking matters. If it is small, explain why that is realistic for your domain.